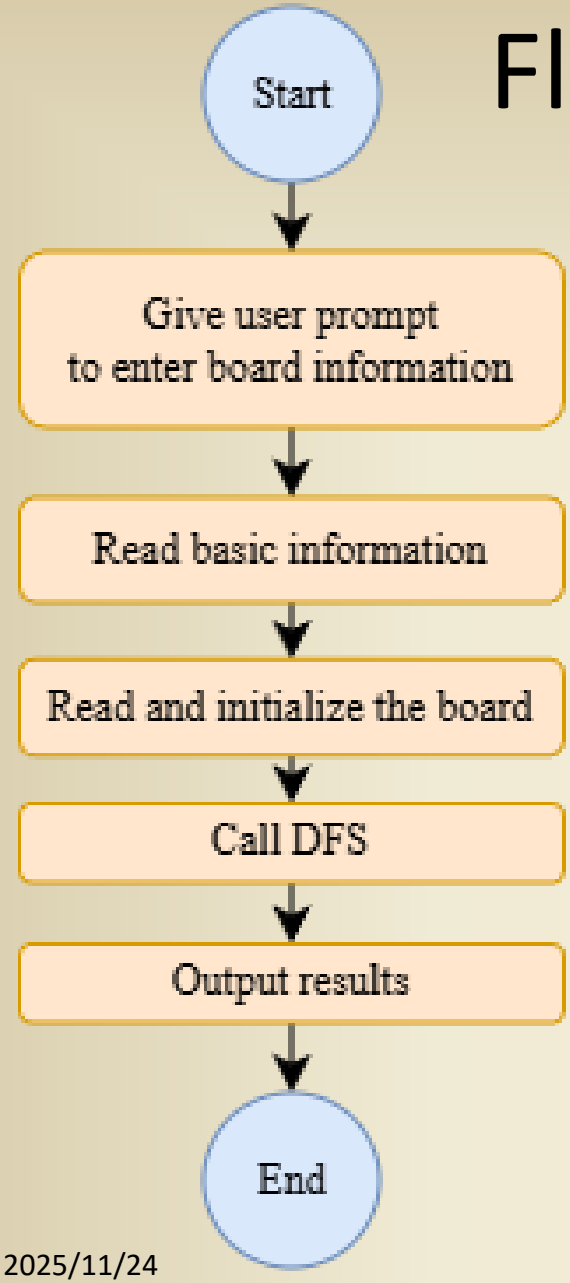
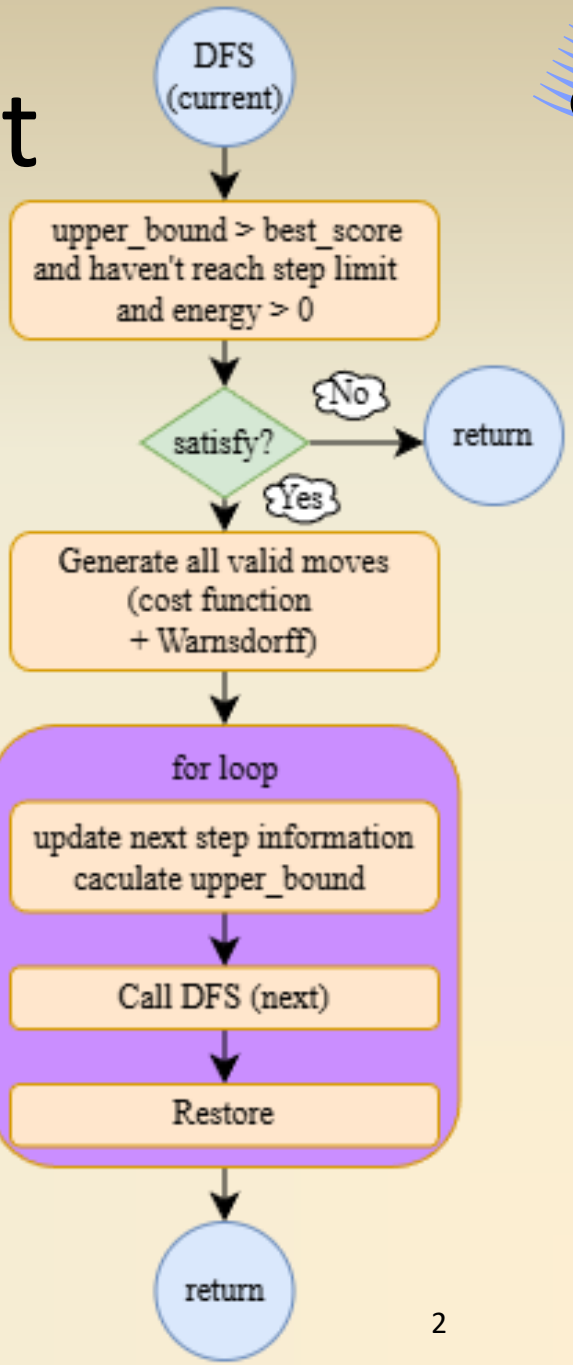
PA2: Knight's Treasure Hunt
Final Explanation

Andy, Yu-Guang Chen
Associate Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw
Slides Credit: TA Tse-Chuan Yang



Flowchart


Start
Give user prompt to enter board information
Read basic information
Read and initialize the board
Call DFS
Output results
End


DFS (current)
upper_bound > best_score and haven't reach step limit and energy > 0
satisfy?
No
return
Yes
Generate all valid moves (cost function + Warnsdorff)
for loop
update next step information caculate upper_bound
Call DFS (next)
Restore
return

Data Structure

- Because the board size and limits are known in advance, we can allocate **fixed-size(20x20)** arrays, and all board grids are kept in global 2D arrays.
- Knight movement directions are stored in constant arrays for convenient access.
- The path is stored using a **2D array(101x2),** representing (x, y) at each step.

```
const int DIRECTION = 8;
const int KnightMoveRow[DIRECTION] = {2, 1, -1, -2, -2, -1, 1, 2};
const int KnightMoveCol[DIRECTION] = {1, 2, 2, 1, -1, -2, -2, -1};
bool is_obstacle[20][20];
bool is_bonus[20][20];
bool is_energy[20][20];
bool is_bomb[20][20];
bool visited[20][20] = {false};
int best_path[101][2];
int cur_path[101][2];
```



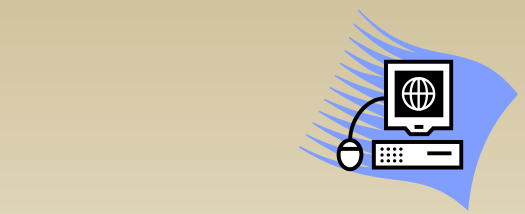
Read Input & Initialize

- The board is read tile-by-tile and each character is mapped into a set of **2D arrays**.
- The starting position ‘S’ is detected during input and stored as first element in current path.
- The visited array is initialized to false, and the start tile is marked true.
- The program **counts the number of remaining grids**, which are later used upper-bound estimation in DFS pruning

```
for (int r = 0; r < N; r++)
    for (int c = 0; c < N; c++)
    {
        char ch;
        cin >> ch;
        if (ch == 'S')
        {
            start_place = {r, c};
            remAvaliableStep--;
        }
        if (ch == '#')
        {
            is_obstacle[r][c] = true;
            remAvaliableStep--;
        }
        if (ch == 'B')
        {
            is_bonus[r][c] = true;
            remB++;
        }
        if (ch == 'E')
        {
            is_energy[r][c] = true;
            remE++;
        }
        if (ch == 'X')
            is_bomb[r][c] = true;
    }
```







Recursion

1. Check the **upper bound**; return immediately if this path cannot beat the current best score.
2. If the current score is better, **update the best solution**.
3. If the **step limit** is reached or **energy is zero**, return.
4. Generate all valid knight moves **(by cost function).**
5. For each move:
   - Compute **next** score and energy
   - Mark visited to the current path
   - Update the number of remaining grids
   - **Recursively call DFS(next)**
   - Backtrack and restore all states





Move Generation

- First check whether each move is inside the board and valid.

```
inline bool in_board(int r, int c) // whether in the board
{
    return (r >= 0 && r < N && c >= 0 && c < N);
}
```

- Determine the **number of available moves** from **next position** (i.e. the degree).
- Evaluate the score gain and energy gain/loss for that move.
- Build a cost function so the knight **prefers** moves with **higher score, better energy recovery, and smaller degree.**

```
int pri = 9 * add_score + 5 * add_energy - 1 * deg;
```



Upper Bound

- Compute an optimistic score bound using reachable steps, possible energy recovery, and all remaining bonuses, and prune if this bound cannot exceed the current best score.
  (Below is a rough upper-bound estimation)

```
// A Rough Upper Bound Estimation
int compute_upper_bound(int step, int energy, int score)
{
    // Remaining usable tiles (not obstacles and not visited)
    int RemStep = min(Step_limit - step, remAvaliableStep);
    // Maximum number of moves possible
    int max_basic_moves = min(RemStep, energy + remE * (E_value));
    // Maximum number of bonus moves possible
    int max_bonus_moves = min(RemStep, energy + remE * (E_value - 1));
    // Maximum additional score from remaining bonuses
    int max_add_score = min(max_bonus_moves, remB) * B_value;
    return score + max_basic_moves + max_add_score;
}
```

Bonus

- Andy expands every 5 steps, so we first generate four **blocking masks** (steps 0–4, 5–9, 10–14, and ≥15).
  Each mask marks the grids that will be blocked by Andy during that step range, allowing us to avoid scanning all ‘A’ repeatedly.
- When generating knight moves, any position that falls inside the mask is filtered out.
- However, at steps 4, 9, and 14 specifically, we must additionally ensure that the **next position** will not be blocked after Andy expands.
- When estimating the upper bound, we just **treat Andy block as obstacles**.



Recursion

- At steps 4, 9, and 14, Andy expands next step, changing which grids remain reachable.
  Thus, the upper bound must be recalculated.

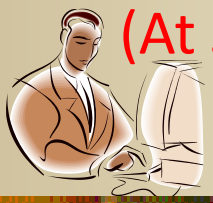
For each move:

- Compute **next** score and energy
- Mark visited to the current path
- Update the number of remaining grids

(At steps **4, 9, and 14**, Andy expands, so we must recompute the remaining usable grids.)

- **Recursively call DFS(next)**
- Backtrack and restore all states

(At steps **4, 9, and 14**, we also restore the remaining grid info.)



Thank you!



